

Declan Sheehan

Cosc 320-001

Dr. Anderson

17 September, 2019

README - Lab 3

Theoretic Time Complexity:

Sort	Best	Worst
Heap Sort	$O(n\log(n))$	$O(n\log(n))$
n=100	200	200
n=1000	3000	3000
n=10000	40000	40000

Approach:

The issue or problem that needed to be solved was: to make a heap of elements in a single array, then sort the entire array, but based off of the structure of the heap. With this in mind, I followed the pseudocode provided to sort the array based off of the max/min heap. Not much else other than this.

Absolute Timing Scale:

The results reflect the absolute timing scale for a heap sort. In this case; $O(n\log(n))$ was the best and worst case for our sort. The results I was given essentially reflect the merge-sort results, so it is safe to say that the results match $O(n\log(n))$ for best and worst case. This is because merge-sort also has $O(n\log(n))$ for best and worst cases. For each array size `n`, the time complexity matches. Assuming I know what a non-asymptotic function is, I believe the graph/results follow closely with the function $n\log(n)$, where $n\log(n)$ is a sublinear function.

Performances:

It is hard to extract which method roughly took longer or shorter than another. This is since technically we do not have enough data points to say that it is statistically significant ($p \leq 0.05$). Also, from the data, some perform better than another in larger n-values, but worse in smaller n-values. Worst & best cases were $O(n\log(n))$.

Code Improvement:

I could make the code more organized. I could also create a function that ran `heapsorting` a few times and take the average of each time to get more accurate data. And lastly, The code could be translated into a lower-level language in order to implement the same commands in quicker and more efficient methods; which would make things even more accurate.